

Assignment 2

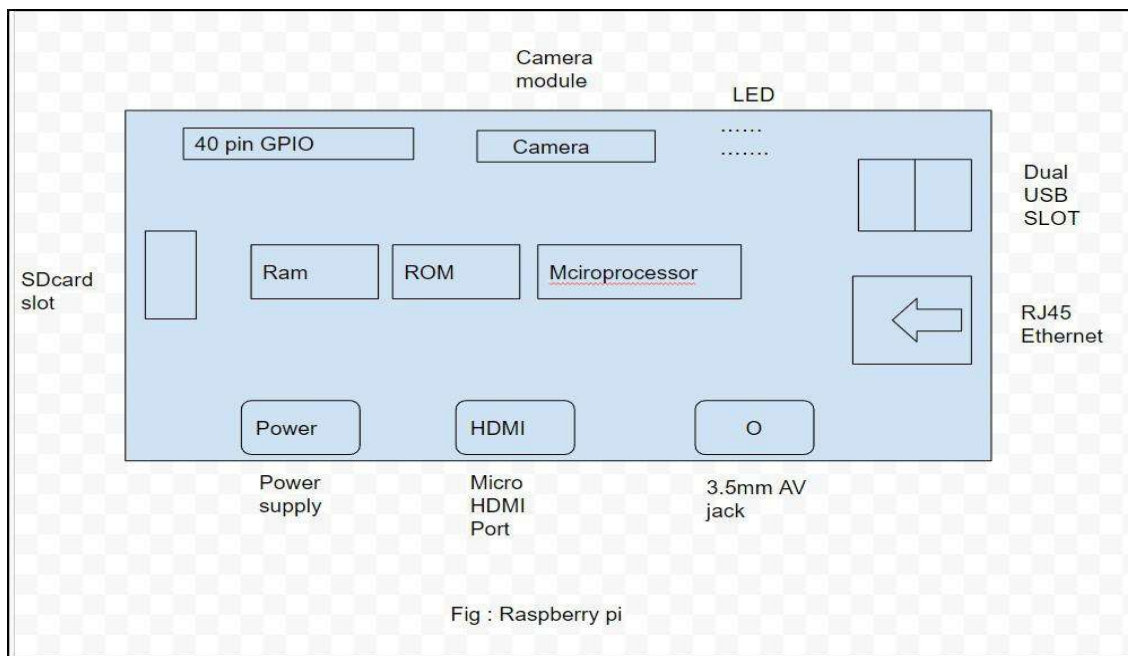
Raspberry Pi Interfaces and Raspberry Pi with Python Programming

This document provides an in-depth explanation of Raspberry Pi hardware interfaces and their integration with Python programming. It covers the evolution of Raspberry Pi, detailed hardware communication methods, example projects, and future trends in its usage.

1. Introduction and History

The Raspberry Pi was developed by the Raspberry Pi Foundation in the UK to promote computer science education. Since its first release in 2012, it has evolved into a powerful platform used in education, research, IoT, robotics, and AI applications.

Major models include:



Model	Year	Processor	RAM	Notable Features
Raspberry Pi 1	2012	700 MHz ARM11	256MB/512MB	First generation
Raspberry Pi 2	2015	900 MHz Quad-Core	1GB	Improved CPU
Raspberry Pi 3	2016	1.2 GHz Quad-Core	1GB	Wi-Fi, Bluetooth
Raspberry Pi 4	2019	1.5 GHz Quad-Core	2-8GB	USB 3.0, Dual HDMI

1. Raspberry Pi Interfaces

The Raspberry Pi's interfaces allow it to connect to and control various devices. These include GPIO pins, serial communication protocols, multimedia outputs, and networking capabilities.

1.1 GPIO (General Purpose Input/Output) Pins

GPIO pins can be set as inputs or outputs, enabling the Pi to interact with external devices such as LEDs, buttons, motors, and sensors.

Raspberry Pi GPIO Layout (40-pin)

Pin 1
Pin 2
Pin 3
Pin 4
Pin 5
Pin 6
Pin 7
Pin 8
Pin 9
Pin 10
Pin 11
Pin 12
Pin 13
Pin 14
Pin 15
Pin 16
Pin 17
Pin 18
Pin 19
Pin 20

1.1 Communication Protocols

Protocol	Wires Used	Speed	Applications
I ² C	2	100 kbps – 3.4 Mbps	Multiple sensors
SPI	4+	Up to 125 Mbps	Displays, ADC/DAC
UART	2	Varies	GPS, Serial Consoles

1. Raspberry Pi with Python Programming

Python is ideal for Raspberry Pi because of its simplicity and vast library support for hardware control. Libraries like RPi.GPIO and gpiozero make it easy to control hardware components directly from code.

1.1 Example: Blinking LED

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
LED = 18
GPIO.setup(LED, GPIO.OUT)
for i in range(10):
    GPIO.output(LED, True)
    time.sleep(0.5)
    GPIO.output(LED, False)
    time.sleep(0.5)
GPIO.cleanup()
```

1.2 Example: Reading Temperature and Humidity

```
import Adafruit_DHT
sensor = Adafruit_DHT.DHT11
pin = 4
humidity, temperature = Adafruit_DHT.read(sensor, pin)
if humidity and temperature:
    print(f"Temp: {temperature}°C Humidity: {humidity}%")
else:
    print("Sensor error")
```

1.3 More Project Ideas

- Smart Home Control
- Weather Monitoring Station
- IoT Device with Cloud Integration
- AI-based Security Camera

2. Advantages and Limitations

Advantages:

- Low cost and small size
- Flexible and programmable
- Wide community support

Limitations:

- Limited processing power compared to desktops
- No built-in storage; uses microSD cards

3. Future Trends of Raspberry Pi

Raspberry Pi is expected to be widely used in AI, edge computing, industrial automation, and low-cost computing solutions for developing nations. Integration with 5G networks and more powerful ARM processors will further enhance its capabilities.

Example Project – Blinking LED (Hello World of Hardware)

```
import Adafruit_DHT

sensor = Adafruit_DHT.DHT11

pin = 4 # GPIO4

humidity, temperature = Adafruit_DHT.read(sensor, pin)

if humidity is not None and temperature is not None:

    print(f"Temperature: {temperature}°C, Humidity: {humidity}%")

else:

    print("Sensor reading failed")
```

output:

ON -> 0.5s pause

OFF -> 0.5s pause

(repeats total 5 times)

Example Project – Sensor Reading

Let's say we have a DHT11 temperature & humidity sensor:

```
import Adafruit_DHT

sensor = Adafruit_DHT.DHT11

pin = 4 # GPIO4

humidity, temperature = Adafruit_DHT.read(sensor, pin)

if humidity is not None and temperature is not None:

    print(f'Temperature: {temperature}°C, Humidity: {humidity}%')

else:

    print("Sensor reading failed")
```

output:

Temperature: 27°C, Humidity: 62%

or, if the sensor fails to return data (due to wiring issues, timing, or sensor initialization problems):

nginx

CopyEdit

Sensor reading failed

- ☐ **What happens:** Reads live weather data.
- ☐ **Learning outcome:** How to fetch and display data from a sensor.

4. Conclusion

Raspberry Pi's combination of diverse interfaces and Python's simplicity enables endless possibilities. From small hobby projects to complex IoT systems, this duo continues to inspire innovation in technology.

